Constraints in MSSQL (SQL Server)

Constraints in SQL Server **enforce rules** on table columns to ensure **data integrity and accuracy**.

□ Types of Constraints in MSSQL

Constraint	Description
PRIMARY KEY	Ensures uniqueness and cannot have `NULL` values
FOREIGN KEY	Links a column to another table's primary key
UNIQUE	Ensures all values in a column are different
NOT NULL	Ensures a column cannot store `NULL` values
CHECK	Defines a condition that must be met for values in a column
DEFAULT	Provides a default value if none is specified

---

1. PRIMARY KEY Constraint

 Ensures **uniqueness** and **not null**.

sql

CopyEdit

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(100),
    Age INT
);
```

 Alternative Syntax:

sql

CopyEdit

```
CREATE TABLE Employees (
    EmployeeID INT,
    Name VARCHAR(100),
    Age INT,
    CONSTRAINT PK_Employee PRIMARY KEY (EmployeeID)  -- Named
Constraint
);
```

□ 2. FOREIGN KEY Constraint

□ Creates a **relationship** between two tables.

sql

CopyEdit

```
CREATE TABLE Departments (
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(100)
);

CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(100),
    DepartmentID INT,
    CONSTRAINT FK_EmployeeDept FOREIGN KEY (DepartmentID)
    REFERENCES Departments(DepartmentID)
);
```

□ **Prevents deletion of referenced rows unless handled properly.**

3. UNIQUE Constraint

 Ensures that all values in a column are **different**.

sql

CopyEdit

```
CREATE TABLE Users (
    UserID INT PRIMARY KEY,
    Email VARCHAR(100) UNIQUE  -- Ensures email is unique
);
```

 Alternative Syntax:

sql

CopyEdit

```
CREATE TABLE Users (
    UserID INT PRIMARY KEY,
    Email VARCHAR(100),
    CONSTRAINT UQ_Email UNIQUE (Email)  -- Named Constraint
);
```

4. NOT NULL Constraint

 Prevents `NULL` values in a column.

sql

CopyEdit

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL, -- Name must have a value
    Age INT
);
```

---

5. CHECK Constraint

 Defines a **custom condition** for a column.

sql

CopyEdit

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(100),
    Age INT CHECK (Age >= 18) -- Ensures age is at least 18
);
```

 Alternative Syntax:

sql

CopyEdit

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(100),
    Age INT,
    CONSTRAINT CK_Age CHECK (Age >= 18)  -- Named Constraint
);
```

□ 6. DEFAULT Constraint

□ Sets a **default value** if none is provided.

sql

CopyEdit

```
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    OrderDate DATETIME DEFAULT GETDATE() -- Defaults to current date
);
```

Managing Constraints

□ Drop a Constraint

sql

CopyEdit

```
ALTER TABLE Employees DROP CONSTRAINT CK_Age;
```

□ Add a Constraint Later

sql

CopyEdit

```
ALTER TABLE Employees ADD CONSTRAINT CK_Age CHECK (Age >= 18);
```

☑ Summary

Constraint	Purpose
PRIMARY KEY	Ensures **uniqueness** and prevents `NULL`
FOREIGN KEY	Maintains **referential integrity** between tables
UNIQUE	Ensures all values in a column are **different**
NOT NULL	Prevents `NULL` values in a column

CHECK	Defines **custom conditions**
DEFAULT	Assigns a **default value** if none is provided

□ Keys in MSSQL (SQL Server)

Keys in SQL Server are **used to establish relationships between tables** and ensure **data integrity**.

□ Types of Keys in MSSQL

Key Type	Description
Primary Key	Ensures **uniqueness** of records in a table, does **not allow NULLs**
Foreign Key	Establishes a **relationship** between two tables
Unique Key	Ensures all values in a column are **unique**, but allows **one NULL**
Composite Key	A **Primary or Unique Key** made up of **multiple columns**
Candidate Key	Any column that can be a **Primary Key**
Alternate Key	A **Candidate Key** that is **not chosen** as the Primary Key

Super Key	A **set of one or more keys** that uniquely identifies a record
Surrogate Key	A **system-generated unique identifier**, usually an **IDENTITY column**

□ 1. PRIMARY KEY (PK)

□ Ensures **uniqueness** and does **not allow NULL values**.

sql

CopyEdit

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(100),
    Age INT
);
```

□ **Alternative Syntax:**

sql

CopyEdit

```
CREATE TABLE Employees (
    EmployeeID INT,
    Name VARCHAR(100),
    Age INT,
```

```
    CONSTRAINT PK_Employee PRIMARY KEY (EmployeeID)  -- Named
Constraint

);
```

☑**Primary Key Rules:**

- Only **one** primary key per table.
- Cannot have **NULL values**.
- Automatically creates a **Clustered Index**.

□ 2. FOREIGN KEY (FK)

□ Establishes a **relationship** between two tables.

sql

CopyEdit

```
CREATE TABLE Departments (

    DepartmentID INT PRIMARY KEY,

    DepartmentName VARCHAR(100)

);


CREATE TABLE Employees (

    EmployeeID INT PRIMARY KEY,

    Name VARCHAR(100),

    DepartmentID INT,

    CONSTRAINT FK_EmployeeDept FOREIGN KEY (DepartmentID)

    REFERENCES Departments(DepartmentID)
```

```
);
```

☑**Foreign Key Rules:**

- A **Foreign Key** in one table refers to a **Primary Key** in another.
- Prevents **deleting referenced data** unless constraints are handled.

□ 3. UNIQUE KEY

□ Ensures **unique** values but **allows one NULL**.

sql

CopyEdit

```
CREATE TABLE Users (
    UserID INT PRIMARY KEY,
    Email VARCHAR(100) UNIQUE  -- Ensures email is unique
);
```

□ **Alternative Syntax:**

sql

CopyEdit

```
CREATE TABLE Users (
    UserID INT PRIMARY KEY,
    Email VARCHAR(100),
    CONSTRAINT UQ_Email UNIQUE (Email)  -- Named Constraint
);
```

☑**Unique Key Rules:**

- Allows **NULL values** (but only **one NULL**).
- Ensures column **values are unique**.
- Can have **multiple unique keys** per table.

□ 4. COMPOSITE KEY

□ A **Primary or Unique Key** with **multiple columns**.

sql

CopyEdit

```
CREATE TABLE Orders (
    OrderID INT,
    ProductID INT,
    Quantity INT,
    PRIMARY KEY (OrderID, ProductID)  -- Composite Primary Key
);
```

☑**Composite Key Rules:**

- Ensures **both columns combined** are unique.
- Used when a **single column isn't enough** to uniquely identify records.

□ 5. CANDIDATE KEY

□ Any column that **could be** a Primary Key.

sql

CopyEdit

```
CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY,  -- Chosen as Primary Key
    Email VARCHAR(100) UNIQUE,   -- Could also be a Primary Key
    PhoneNumber VARCHAR(20) UNIQUE
);
```

☑**Candidate Key Rules:**

- Any column that can be **used as a unique identifier**.
- The **Primary Key is chosen** from Candidate Keys.

□ 6. ALTERNATE KEY

□ A **Candidate Key** that is **not chosen** as the Primary Key.

sql

CopyEdit

```
CREATE TABLE Vehicles (
    VehicleID INT PRIMARY KEY,
    LicensePlate VARCHAR(20) UNIQUE  -- Alternate Key (Candidate Key not chosen)
);
```

☑**Alternate Key Rules:**

- A **Candidate Key** that **wasn't chosen** as the Primary Key.

□ 7. SUPER KEY

 A **set of one or more keys** that uniquely identifies a row.

sql

CopyEdit

```
CREATE TABLE Orders (
    OrderID INT,
    CustomerID INT,
    OrderDate DATETIME,
    PRIMARY KEY (OrderID)  -- Super Key (OrderID alone is enough)
);
```

☑**Super Key Rules:**

- Includes **Primary Keys, Candidate Keys, and any additional columns**.

---

8. SURROGATE KEY

 A **system-generated unique identifier**, usually an **IDENTITY column**.

sql

CopyEdit

```
CREATE TABLE Transactions (
    TransactionID INT IDENTITY(1,1) PRIMARY KEY,
    Amount DECIMAL(10,2),
    TransactionDate DATETIME DEFAULT GETDATE()
);
```

☑**Surrogate Key Rules:**

- Does **not have business meaning**.
- Commonly used as **auto-incremented IDs**.

☑Summary of Keys in MSSQL

Key Type	Description
Primary Key	Ensures **uniqueness**, **no NULL values**
Foreign Key	Links tables, maintains **referential integrity**
Unique Key	Ensures **unique values**, allows **one NULL**
Composite Key	Primary or Unique Key with **multiple columns**
Candidate Key	Any column that **could be a Primary Key**
Alternate Key	A **Candidate Key** that wasn't chosen as the Primary Key
Super Key	A **set of one or more keys** that uniquely identifies a record
Surrogate Key	A **system-generated** unique identifier (e.g., `IDENTITY(1,1)`)

Would you like **examples for querying data** using these keys? □

—----------

□ Data Control Language (DCL) in MSSQL

DCL (Data Control Language) in SQL Server is used to **control access** to the database by granting and revoking permissions.

□ DCL Commands

Command	Description
`GRANT`	Gives a user **permission** to perform a specific operation (e.g., SELECT, INSERT)
`REVOKE`	**Removes** a previously granted permission from a user
`DENY`	Explicitly **blocks** a user from performing an operation

---

□ 1. `GRANT` – Assign Permissions

□ Grants **specific permissions** to a user or role.

sql

CopyEdit

```
GRANT SELECT, INSERT ON Employees TO John;
```

✅ **John can now** `SELECT` and `INSERT` data in the `Employees` table.

 Grant All Permissions

sql

CopyEdit

```
GRANT ALL ON Employees TO John;
```

✅ John gets **all** privileges on the `Employees` table.

 Grant Database-Level Permissions

sql

CopyEdit

```
GRANT CREATE TABLE TO John;
```

✅ John can now create tables.

---

2. `REVOKE` – Remove Permissions

 Removes previously granted permissions from a user.

sql

CopyEdit

```
REVOKE INSERT ON Employees FROM John;
```

✅ John **loses** `INSERT` permission but **can still SELECT**.

 Revoke All Permissions

sql

CopyEdit

```
REVOKE ALL ON Employees FROM John;
```

☑John **loses all** privileges on `Employees`.

□ 3. DENY – Explicitly Block Access

□ **Blocks a user from performing an action**, even if they have permission through a role.

sql

CopyEdit

```
DENY DELETE ON Employees TO John;
```

☑John is **explicitly denied** `DELETE`, even if part of a role with `DELETE` permission.

□ **Remove DENY**

sql

CopyEdit

```
REVOKE DELETE ON Employees FROM John;
```

☑Removes the explicit denial, allowing inheritance from roles.

□ Managing Users and Roles

□ Create a New User

sql

CopyEdit

```
CREATE LOGIN John WITH PASSWORD = 'SecurePass123';

CREATE USER John FOR LOGIN John;
```

□ Assign User to a Role

sql

CopyEdit

```
ALTER ROLE db_datareader ADD MEMBER John;
```

☑John can now read all tables.

□ Drop a User

sql

CopyEdit

```
DROP USER John;

DROP LOGIN John;
```

☑Summary

Command	Purpose
GRANT	Allows a user to perform an action
REVOKE	Removes a granted permission

DENY	Explicitly blocks a user from an action

Would you like an **example with roles** or **advanced security settings**?

—----------

SQL Joins in MSSQL with Examples

In **Microsoft SQL Server (MSSQL)**, **joins** are used to combine records from two or more tables based on a related column.

Types of Joins in MSSQL

Join Type	Description
INNER JOIN	Returns only matching rows from both tables
LEFT JOIN (LEFT OUTER JOIN)	Returns all rows from the left table + matching rows from the right table
RIGHT JOIN (RIGHT OUTER JOIN)	Returns all rows from the right table + matching rows from the left table
FULL JOIN (FULL OUTER JOIN)	Returns all rows when there is a match in either table
CROSS JOIN	Returns the Cartesian product of both tables

SELF JOIN	Joins a table with itself

□ Example Setup: Create Sample Tables

sql

CopyEdit

```
-- Creating Employees Table
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(50),
    DepartmentID INT
);

-- Creating Departments Table
CREATE TABLE Departments (
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(50)
);

-- Insert Data into Employees
INSERT INTO Employees (EmployeeID, Name, DepartmentID) VALUES
(1, 'Alice', 1),
```

```
(2, 'Bob', 2),

(3, 'Charlie', 3),

(4, 'David', NULL);


-- Insert Data into Departments

INSERT INTO Departments (DepartmentID, DepartmentName) VALUES

(1, 'HR'),

(2, 'IT'),

(3, 'Finance'),

(4, 'Marketing');
```

□ 1. INNER JOIN (Only Matching Rows)

Returns only rows where there is a match in both tables.

sql

CopyEdit

```
SELECT Employees.EmployeeID, Employees.Name,
Departments.DepartmentName

FROM Employees

INNER JOIN Departments ON Employees.DepartmentID =
Departments.DepartmentID;
```

☑**Result:**

markdown

CopyEdit

```
EmployeeID | Name    | DepartmentName
-------------------------------------
1          | Alice   | HR
2          | Bob     | IT
3          | Charlie | Finance
```

 `David` is not included because he has no `DepartmentID` (NULL).

---

2. LEFT JOIN (All from Left Table + Matches from Right Table)

Returns all employees, even if they don't have a department.

sql

CopyEdit

```
SELECT Employees.EmployeeID, Employees.Name,
Departments.DepartmentName
FROM Employees
LEFT JOIN Departments ON Employees.DepartmentID =
Departments.DepartmentID;
```

Result:

markdown

CopyEdit

```
EmployeeID | Name    | DepartmentName
```

```
-------------------------------------

1           | Alice   | HR

2           | Bob     | IT

3           | Charlie | Finance

4           | David   | NULL
```

 `David` is included, but since he has no department, `DepartmentName` is `NULL`.

---

3. RIGHT JOIN (All from Right Table + Matches from Left Table)

Returns all departments, even if they don't have employees.

sql

CopyEdit

```
SELECT Employees.EmployeeID, Employees.Name,
Departments.DepartmentName

FROM Employees

RIGHT JOIN Departments ON Employees.DepartmentID =
Departments.DepartmentID;
```

Result:

pgsql

CopyEdit

```
EmployeeID | Name    | DepartmentName

-------------------------------------
```

```
1           | Alice   | HR

2           | Bob     | IT

3           | Charlie | Finance

NULL        | NULL    | Marketing
```

□ The `Marketing` department has **no employees**, so `EmployeeID` and `Name` are `NULL`.

□ 4. FULL OUTER JOIN (All Records from Both Tables)

Returns all employees and all departments, matching where possible.

sql

CopyEdit

```
SELECT Employees.EmployeeID, Employees.Name,
Departments.DepartmentName

FROM Employees

FULL OUTER JOIN Departments ON Employees.DepartmentID =
Departments.DepartmentID;
```

☑**Result:**

pgsql

CopyEdit

```
EmployeeID | Name    | DepartmentName

-------------------------------------

1           | Alice   | HR

2           | Bob     | IT
```

```
3           | Charlie | Finance
4           | David   | NULL
NULL        | NULL    | Marketing
```

 Includes unmatched records from both tables.

□ 5. CROSS JOIN (Cartesian Product)

Returns every combination of employees and departments.

sql

CopyEdit

```
SELECT Employees.Name, Departments.DepartmentName
FROM Employees
CROSS JOIN Departments;
```

☑**Result (4 Employees × 4 Departments = 16 rows):**

markdown

CopyEdit

```
Name      | DepartmentName
-------------------------
Alice     | HR
Alice     | IT
Alice     | Finance
Alice     | Marketing
```

```
Bob      | HR

Bob      | IT

Bob      | Finance

Bob      | Marketing

...
```

 Every employee is combined with every department.

---

6. SELF JOIN (Joining a Table with Itself)

Find employees who work in the same department as another employee.

sql

CopyEdit

```
SELECT e1.Name AS Employee1, e2.Name AS Employee2, d.DepartmentName

FROM Employees e1

JOIN Employees e2 ON e1.DepartmentID = e2.DepartmentID AND
e1.EmployeeID <> e2.EmployeeID

JOIN Departments d ON e1.DepartmentID = d.DepartmentID;
```

☑**Finds employees in the same department (excluding themselves).**

☑Summary of SQL Joins

Join Type	Matching Behavior

INNER JOIN	Only matching rows from both tables
LEFT JOIN	All rows from left table + matching from right
RIGHT JOIN	All rows from right table + matching from left
FULL JOIN	All rows from both tables, matching where possible
CROSS JOIN	Every row from first table with every row from second
SELF JOIN	Joins a table with itself

—--

Subqueries in MSSQL with Examples

A **subquery** in **Microsoft SQL Server (MSSQL)** is a query inside another query. It is used to fetch data that will be used in the main query.

□ Types of Subqueries in SQL

Type	Description
Single-Row Subquery	Returns **one value** (e.g., `=`, `<`, `>`)

Multi-Row Subquery	Returns **multiple values** (used with `IN`, `ANY`, `ALL`)
Correlated Subquery	Subquery depends on the main query (row-by-row processing)
Nested Subquery	A subquery inside another subquery

□ Example Setup: Create Sample Tables

sql
CopyEdit

```
-- Create Employees Table
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(50),
    DepartmentID INT,
    Salary INT
);

-- Create Departments Table
CREATE TABLE Departments (
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(50)
);

-- Insert Data into Employees
INSERT INTO Employees (EmployeeID, Name, DepartmentID, Salary) VALUES
(1, 'Alice', 1, 60000),
(2, 'Bob', 2, 50000),
(3, 'Charlie', 1, 70000),
(4, 'David', 3, 45000),
(5, 'Eve', 2, 65000);

-- Insert Data into Departments
INSERT INTO Departments (DepartmentID, DepartmentName) VALUES
(1, 'HR'),
(2, 'IT'),
(3, 'Finance');
```

□ 1. Single-Row Subquery

Find employees who have the highest salary.

sql
CopyEdit

```
SELECT Name, Salary
FROM Employees
WHERE Salary = (SELECT MAX(Salary) FROM Employees);
```

☑**Result:**

markdown
CopyEdit

```
Name     | Salary
-----------------
Charlie  | 70000
```

□ The subquery returns the **highest salary**, and the main query finds the employee with that salary.

□ 2. Multi-Row Subquery

Find employees who work in departments with IT or HR.

sql
CopyEdit

```
SELECT Name, DepartmentID
FROM Employees
WHERE DepartmentID IN (SELECT DepartmentID FROM Departments WHERE
DepartmentName IN ('IT', 'HR'));
```

☑**Result:**

markdown

CopyEdit

```
Name    | DepartmentID
-----------------------
Alice   | 1
Bob     | 2
Charlie | 1
Eve     | 2
```

 The subquery fetches `DepartmentID` for **IT and HR**, and the main query filters employees.

□ 3. Correlated Subquery

Find employees whose salary is higher than the average salary in their department.

sql
CopyEdit

```
SELECT Name, Salary, DepartmentID
FROM Employees e1
WHERE Salary > (SELECT AVG(Salary) FROM Employees e2 WHERE
e1.DepartmentID = e2.DepartmentID);
```

☑**Result:**

markdown
CopyEdit

```
Name    | Salary | DepartmentID
--------------------------------
Charlie | 70000  | 1
Eve     | 65000  | 2
```

 The subquery **depends on each row of the main query**, calculating the **average salary per department**.

□ 4. Nested Subquery

Find employees who work in the department where 'Charlie' works.

sql
CopyEdit

```
SELECT Name
FROM Employees
WHERE DepartmentID = (
    SELECT DepartmentID FROM Employees WHERE Name = 'Charlie'
);
```

☑**Result:**

markdown
CopyEdit

```
Name
-------
Alice
Charlie
```

 The **inner query** finds `Charlie`'s `DepartmentID`, and the **outer query** finds employees in that department.

---

5. Subquery in `SELECT` Clause

Display each employee's salary along with the department name.

sql
CopyEdit

```
SELECT e.Name, e.Salary,
       (SELECT d.DepartmentName FROM Departments d WHERE
d.DepartmentID = e.DepartmentID) AS Department
FROM Employees e;
```

☑**Result:**

markdown
CopyEdit

```
Name     | Salary | Department
```

```
----------------------------
Alice    | 60000  | HR
Bob      | 50000  | IT
Charlie  | 70000  | HR
David    | 45000  | Finance
Eve      | 65000  | IT
```

 The subquery retrieves the `DepartmentName` based on the `DepartmentID`.

---

6. Subquery in `FROM` Clause (Derived Table)

Find the average salary of each department and filter departments with an average salary > 55000.

sql
CopyEdit

```
SELECT DepartmentID, AvgSalary
FROM (SELECT DepartmentID, AVG(Salary) AS AvgSalary FROM Employees
GROUP BY DepartmentID) AS DeptAvg
WHERE AvgSalary > 55000;
```

Result:

markdown
CopyEdit

```
DepartmentID | AvgSalary
-----------------------
1            | 65000
2            | 57500
```

 The **inner query** calculates the average salary, and the **outer query** filters results.

Summary of Subquery Types

Subquery Type	Usage Example
Single-Row Subquery	`WHERE Salary = (SELECT MAX(Salary) FROM Employees);`
Multi-Row Subquery	`WHERE DepartmentID IN (SELECT DepartmentID FROM Departments WHERE DepartmentName = 'IT');`
Correlated Subquery	`WHERE Salary > (SELECT AVG(Salary) FROM Employees e2 WHERE e1.DepartmentID = e2.DepartmentID);`
Nested Subquery	`WHERE DepartmentID = (SELECT DepartmentID FROM Employees WHERE Name = 'Charlie');`
Subquery in `SELECT` Clause	`SELECT Name, (SELECT DepartmentName FROM Departments WHERE ...) AS Department;`
Subquery in `FROM` Clause	`SELECT * FROM (SELECT AVG(Salary) FROM Employees GROUP BY DepartmentID) AS Sub;`

—----------

Aggregate Functions in MSSQL with Examples

In **Microsoft SQL Server (MSSQL)**, **aggregate functions** perform a calculation on a set of values and return a single value. They are commonly used in **GROUP BY** and **HAVING** clauses.

□ Common Aggregate Functions in MSSQL

Function	Description
`SUM()`	Returns the total sum of a numeric column
`AVG()`	Returns the average value of a numeric column
`COUNT()`	Returns the number of rows in a column
`MIN()`	Returns the smallest value in a column
`MAX()`	Returns the largest value in a column
`STDEV()`	Returns the standard deviation of a column
`VAR()`	Returns the variance of a column

□ Example Setup: Create Sample Table

sql
CopyEdit

```
-- Create Employees Table
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(50),
    Department VARCHAR(50),
    Salary INT
);

-- Insert Data
INSERT INTO Employees (EmployeeID, Name, Department, Salary) VALUES
(1, 'Alice', 'HR', 60000),
(2, 'Bob', 'IT', 50000),
(3, 'Charlie', 'HR', 70000),
(4, 'David', 'Finance', 45000),
(5, 'Eve', 'IT', 65000);
```

□ 1. `SUM()` - Total Sum

Find the total salary of all employees.

sql
CopyEdit

```
SELECT SUM(Salary) AS TotalSalary FROM Employees;
```

☑Result:

markdown
CopyEdit

```
TotalSalary
-----------
290000
```

□ Adds up all salaries.

□ 2. `AVG()` - Average Value

Find the average salary of all employees.

sql
CopyEdit

```
SELECT AVG(Salary) AS AverageSalary FROM Employees;
```

☑Result:

markdown
CopyEdit

```
AverageSalary
-------------
58000
```

□ Computes the average salary.

□ 3. `COUNT()` - Number of Rows

Find the total number of employees.

sql
CopyEdit

```
SELECT COUNT(*) AS TotalEmployees FROM Employees;
```

☑Result:

markdown
CopyEdit

```
TotalEmployees
--------------
5
```

□ Counts the number of rows.

□ 4. `MIN()` and `MAX()` - Minimum & Maximum

Find the lowest and highest salary.

sql
CopyEdit

```
SELECT MIN(Salary) AS MinSalary, MAX(Salary) AS MaxSalary FROM
Employees;
```

☑Result:

markdown
CopyEdit

```
MinSalary | MaxSalary
----------------------
45000     | 70000
```

 Finds the **smallest** and **largest** salaries.

---

5. `GROUP BY` with Aggregate Functions

Find the total salary per department.

sql
CopyEdit

```
SELECT Department, SUM(Salary) AS TotalSalary
FROM Employees
GROUP BY Department;
```

Result:

markdown
CopyEdit

```
Department | TotalSalary
------------------------
HR         | 130000
IT         | 115000
Finance    | 45000
```

 Groups by `Department` and calculates the sum.

---

6. `HAVING` with Aggregate Functions

Find departments where the total salary is greater than 100,000.

sql
CopyEdit

```
SELECT Department, SUM(Salary) AS TotalSalary
FROM Employees
GROUP BY Department
HAVING SUM(Salary) > 100000;
```

☑ **Result:**

markdown
CopyEdit

```
Department | TotalSalary
-----------------------
HR         | 130000
IT         | 115000
```

 Filters results after grouping.

---

7. `STDEV()` and `VAR()` - Standard Deviation & Variance

Find the standard deviation and variance of salaries.

sql
CopyEdit

```
SELECT STDEV(Salary) AS SalaryStdDev, VAR(Salary) AS SalaryVariance
FROM Employees;
```

☑ **Result:**

markdown
CopyEdit

```
SalaryStdDev | SalaryVariance
----------------------------
9627.42      | 92666666.67
```

 Measures **salary dispersion**.

☑ Summary of Aggregate Functions

Function	Use Case Example

`SUM()`	`SUM(Salary)` to get total salary
`AVG()`	`AVG(Salary)` to get average salary
`COUNT()`	`COUNT(*)` to count employees
`MIN()`	`MIN(Salary)` to find the lowest salary
`MAX()`	`MAX(Salary)` to find the highest salary
`STDEV()`	`STDEV(Salary)` to calculate salary deviation
`VAR()`	`VAR(Salary)` to calculate salary variance

—----------

Stored Procedure in MSSQL with Examples

A **Stored Procedure** in **Microsoft SQL Server (MSSQL)** is a **precompiled collection of SQL statements** stored in the database. It allows reusability, improves performance, and enhances security by restricting direct access to tables.

□ Benefits of Stored Procedures

☑**Encapsulation** – Groups SQL logic into a reusable block
☑**Performance** – Precompiled for faster execution
☑**Security** – Restricts direct access to tables
☑**Reusability** – Can be executed multiple times with different parameters
☑**Maintainability** – Reduces redundancy in queries

□ Syntax of a Stored Procedure

sql
CopyEdit

```
CREATE PROCEDURE ProcedureName
AS
BEGIN
    -- SQL Statements
END;
```

□ Example 1: Simple Stored Procedure

Create a stored procedure to fetch all employees.

sql
CopyEdit

```
CREATE PROCEDURE GetAllEmployees
AS
BEGIN
    SELECT * FROM Employees;
END;
```

☑**Execute the procedure:**

sql
CopyEdit

```
EXEC GetAllEmployees;
```

□ Returns all employee records.

□ Example 2: Stored Procedure with Parameters

Fetch employees from a specific department.

sql
CopyEdit

```
CREATE PROCEDURE GetEmployeesByDepartment
    @Department VARCHAR(50)
AS
BEGIN
    SELECT * FROM Employees WHERE Department = @Department;
END;
```

☑ **Execute the procedure with a parameter:**

sql
CopyEdit

```
EXEC GetEmployeesByDepartment @Department = 'IT';
```

 Fetches employees in the **IT** department.

Example 3: Stored Procedure with Output Parameter

Get the total salary of a department.

sql
CopyEdit

```
CREATE PROCEDURE GetTotalSalaryByDepartment
    @Department VARCHAR(50),
    @TotalSalary INT OUTPUT
AS
BEGIN
    SELECT @TotalSalary = SUM(Salary) FROM Employees WHERE Department
= @Department;
END;
```

☑ **Execute with an output parameter:**

sql
CopyEdit

```
DECLARE @Salary INT;
EXEC GetTotalSalaryByDepartment @Department = 'HR', @TotalSalary =
@Salary OUTPUT;
```

```
PRINT @Salary;
```

 Displays the **total salary** for **HR**.

Example 4: Stored Procedure with IF-ELSE

Check if an employee exists before inserting.

sql
CopyEdit

```
CREATE PROCEDURE AddEmployee
    @EmployeeID INT,
    @Name VARCHAR(50),
    @Department VARCHAR(50),
    @Salary INT
AS
BEGIN
    IF EXISTS (SELECT 1 FROM Employees WHERE EmployeeID = @EmployeeID)
        PRINT 'Employee already exists!';
    ELSE
        INSERT INTO Employees (EmployeeID, Name, Department, Salary)
        VALUES (@EmployeeID, @Name, @Department, @Salary);
END;
```

☑**Execute the procedure:**

sql
CopyEdit

```
EXEC AddEmployee @EmployeeID = 6, @Name = 'Frank', @Department = 'HR',
@Salary = 55000;
```

 Adds an employee **only if they don’t exist**.

Example 5: Stored Procedure with Transactions

Insert an employee with rollback on failure.

sql
CopyEdit

```
CREATE PROCEDURE InsertEmployeeWithTransaction
    @EmployeeID INT,
    @Name VARCHAR(50),
    @Department VARCHAR(50),
    @Salary INT
AS
BEGIN
    BEGIN TRANSACTION;

    BEGIN TRY
        INSERT INTO Employees (EmployeeID, Name, Department, Salary)
        VALUES (@EmployeeID, @Name, @Department, @Salary);

        COMMIT TRANSACTION; -- Commit on success
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION; -- Rollback on failure
        PRINT 'Transaction Failed!';
    END CATCH;
END;
```

☑**Execute the procedure:**

sql
CopyEdit

```
EXEC InsertEmployeeWithTransaction @EmployeeID = 7, @Name = 'Grace',
@Department = 'Finance', @Salary = 62000;
```

 Ensures data consistency by rolling back on failure.

□ Example 6: Alter and Drop Stored Procedure

Modify an existing stored procedure:

sql
CopyEdit

```
ALTER PROCEDURE GetAllEmployees
AS
BEGIN
    SELECT EmployeeID, Name, Department FROM Employees;
END;
```

 Modifies the stored procedure.

Drop a stored procedure:

sql
CopyEdit

```
DROP PROCEDURE GetAllEmployees;
```

 Deletes the stored procedure.

☑ Summary of Stored Procedures

Type	Example
Simple Procedure	`CREATE PROCEDURE GetAllEmployees AS SELECT * FROM Employees;`
With Parameters	`CREATE PROCEDURE GetEmployeesByDepartment @Department VARCHAR(50) AS ...`
With Output Parameters	`CREATE PROCEDURE GetTotalSalaryByDepartment @Department VARCHAR(50), @TotalSalary INT OUTPUT AS ...`
With IF-ELSE	`CREATE PROCEDURE AddEmployee AS BEGIN IF EXISTS (...) ELSE INSERT ... END;`
With Transactions	`BEGIN TRANSACTION; TRY { INSERT ... COMMIT; } CATCH { ROLLBACK; }`

—----------

Functions in MSSQL with Examples

In **Microsoft SQL Server (MSSQL)**, a **function** is a database object that **returns a value** and can be used in queries like built-in functions. Unlike stored procedures, functions must return a value and **cannot modify database state (INSERT, UPDATE, DELETE, etc.).**

□ Types of Functions in MSSQL

Function Type	Description
Scalar Function	Returns a **single value** (e.g., `GETDATE()`, `LEN()`)
Table-Valued Function (TVF)	Returns a **table** (like a `SELECT` query)
System Functions	Built-in functions like `COUNT()`, `SUM()`, `GETDATE()`
Aggregate Functions	Operates on multiple rows (`AVG()`, `MAX()`, `MIN()`, `SUM()`)

□ 1. Scalar Functions (Returns a Single Value)

Syntax:

sql

CopyEdit

```
CREATE FUNCTION FunctionName (@param DataType)
RETURNS ReturnType
AS
BEGIN
    -- Computation
    RETURN value;
END;
```

Example: Get Employee Bonus (10% of Salary)

sql

CopyEdit

```
CREATE FUNCTION GetBonus (@Salary INT)
RETURNS INT
AS
BEGIN
    RETURN (@Salary * 0.10);  -- 10% Bonus Calculation
END;
```

☑**Execute Function:**

sql

CopyEdit

```
SELECT dbo.GetBonus(50000) AS Bonus;
```

☑**Result:**

markdown

CopyEdit

```
Bonus
-----
5000
```

 This function **returns a bonus amount** based on salary.

---

2. Table-Valued Function (Returns a Table)

Syntax:

sql

CopyEdit

```
CREATE FUNCTION FunctionName (@param DataType)
RETURNS TABLE
AS
RETURN
(
```

```
    SELECT Columns FROM Table WHERE Condition
);
```

Example: Get Employees by Department

sql

CopyEdit

```
CREATE FUNCTION GetEmployeesByDepartment (@Department VARCHAR(50))
RETURNS TABLE
AS
RETURN
(
    SELECT EmployeeID, Name, Salary FROM Employees WHERE Department = @Department
);
```

☑Execute Function:

sql

CopyEdit

```
SELECT * FROM dbo.GetEmployeesByDepartment('IT');
```

☑Result:

pgsql

CopyEdit

```
EmployeeID | Name  | Salary
```

```
-----------|------ |-------
2          | Bob   | 50000
5          | Eve   | 65000
```

 This function **returns employees from a specific department.**

---

3. Inline Table-Valued Function (More Efficient Table Function)

Example: Return Employees Earning More Than a Given Salary

sql

CopyEdit

```
CREATE FUNCTION GetHighPaidEmployees (@MinSalary INT)
RETURNS TABLE
AS
RETURN
(
    SELECT EmployeeID, Name, Salary FROM Employees WHERE Salary > @MinSalary
);
```

Execute Function:

sql

CopyEdit

```
SELECT * FROM dbo.GetHighPaidEmployees(55000);
```

☑**Result:**

pgsql

CopyEdit

```
EmployeeID | Name    | Salary
-----------|---------|-------
1          | Alice   | 60000
3          | Charlie| 70000
5          | Eve     | 65000
```

 This function **returns employees who earn more than a given salary.**

---

4. System Functions (Built-in Functions in MSSQL)

Function	Description
`GETDATE()`	Returns the current date and time
`LEN(column)`	Returns the length of a string

`UPPER(column)`	Converts text to uppercase
`LOWER(column)`	Converts text to lowercase
`ROUND(value, decimals)`	Rounds a number
`ISNULL(column, value)`	Replaces NULL with a value
`CONVERT(DATETIME, column)`	Converts data types

☑**Example:**

sql

CopyEdit

```
SELECT GETDATE() AS CurrentDate, UPPER('hello') AS UppercaseText;
```

☑**Result:**

diff

CopyEdit

```
CurrentDate             | UppercaseText
------------------------|--------------
2025-01-30 10:30:00.000 | HELLO
```

 Returns **current date/time** and converts text to **uppercase**.

---

5. Aggregate Functions (Used in GROUP BY)

Function	Example Usage
`SUM()`	`SUM(Salary)` to get total salary
`AVG()`	`AVG(Salary)` to get average salary
`COUNT()`	`COUNT(*)` to count rows
`MIN()`	`MIN(Salary)` to find lowest salary
`MAX()`	`MAX(Salary)` to find highest salary

Example:

sql

CopyEdit

```
SELECT Department, AVG(Salary) AS AvgSalary FROM Employees GROUP BY Department;
```

☑**Result:**

diff

CopyEdit

```
Department | AvgSalary
-----------|-----------
HR         | 65000
IT         | 57500
Finance    | 45000
```

 This query **groups employees by department and calculates their average salary.**

---

6. Alter & Drop Functions

☑**Modify a function (`ALTER FUNCTION`)**

sql

CopyEdit

```
ALTER FUNCTION GetBonus (@Salary INT)
RETURNS INT
AS
BEGIN
    RETURN (@Salary * 0.15);  -- Changed from 10% to 15%
END;
```

☑ Delete a function (`DROP FUNCTION`)

sql

CopyEdit

```
DROP FUNCTION GetBonus;
```

☑ Summary of Functions

Function Type	Example
Scalar Function	`CREATE FUNCTION GetBonus(@Salary INT) RETURNS INT AS BEGIN RETURN (@Salary * 0.10); END;`
Table-Valued Function	`CREATE FUNCTION GetEmployeesByDepartment(@Department VARCHAR(50)) RETURNS TABLE AS RETURN (SELECT * FROM Employees WHERE Department = @Department);`
System Function	`SELECT GETDATE(), UPPER(Name) FROM Employees;`
Aggregate Function	`SELECT COUNT(*) FROM Employees;`

—----------

Views in MSSQL with Examples

A **View** in **Microsoft SQL Server (MSSQL)** is a **virtual table** that represents the result of a SQL query. Views **do not store data**; they simply **store queries** and fetch the data dynamically from the underlying tables.

□ Why Use Views?

☑**Simplifies complex queries** – Encapsulates multiple joins and aggregations.
☑**Enhances security** – Restricts access to specific columns and rows.
☑**Improves maintainability** – Centralizes commonly used queries.
☑**Provides abstraction** – Hides table structure complexity.
☑**Enables performance optimization** – Improves query execution in some cases.

□ Syntax of Creating a View

sql

CopyEdit

```
CREATE VIEW ViewName AS

SELECT column1, column2, ...

FROM TableName

WHERE condition;
```

□ 1. Simple View (Fetching Specific Columns)

Example: Create a View for Employee Names and Salaries

sql

CopyEdit

```
CREATE VIEW EmployeeSalaries AS

SELECT EmployeeID, Name, Salary

FROM Employees;
```

☑Query the view:

sql

CopyEdit

```
SELECT * FROM EmployeeSalaries;
```

☑Result:

pgsql

CopyEdit

```
EmployeeID | Name    | Salary

-----------|---------|-------

1          | Alice   | 60000

2          | Bob     | 50000

3          | Charlie| 70000
```

 The view **fetches EmployeeID, Name, and Salary from the Employees table**.

---

2. View with WHERE Condition (Filtered View)

Example: Create a View for IT Department Employees

sql

CopyEdit

```
CREATE VIEW IT_Employees AS

SELECT EmployeeID, Name, Salary

FROM Employees

WHERE Department = 'IT';
```

☑Query the view:

sql

CopyEdit

```
SELECT * FROM IT_Employees;
```

☑Result:

pgsql

CopyEdit

```
EmployeeID | Name   | Salary

-----------|--------|-------

2          | Bob    | 50000

5          | Eve    | 65000
```

 The view **filters employees who belong to the IT department**.

---

3. View with Join (Combining Multiple Tables)

Example: Create a View for Employee and Department Details

sql

CopyEdit

```
CREATE VIEW EmployeeDetails AS

SELECT e.EmployeeID, e.Name, e.Salary, d.DepartmentName

FROM Employees e

JOIN Departments d ON e.DepartmentID = d.DepartmentID;
```

☑Query the view:

sql

CopyEdit

```
SELECT * FROM EmployeeDetails;
```

☑Result:

pgsql

CopyEdit

```
EmployeeID | Name   | Salary | DepartmentName

-----------|--------|--------|---------------

1          | Alice  | 60000  | HR

2          | Bob    | 50000  | IT

3          | Charlie| 70000  | Finance
```

 The view **combines Employees and Departments table using a JOIN**.

□ 4. View with Aggregation (Using GROUP BY)

Example: View to Get Average Salary by Department

sql

CopyEdit

```
CREATE VIEW AvgSalaryByDepartment AS

SELECT Department, AVG(Salary) AS AvgSalary

FROM Employees

GROUP BY Department;
```

☑Query the view:

sql

CopyEdit

```
SELECT * FROM AvgSalaryByDepartment;
```

☑Result:

diff

CopyEdit

```
Department | AvgSalary

-----------|-----------

HR         | 65000

IT         | 57500

Finance    | 45000
```

 The view **groups employees by department and calculates their average salary**.

---

5. Updating Data Through Views (Updatable Views)

Example: Update Employee Salary Using View

sql

CopyEdit

```
CREATE VIEW UpdateEmployeeSalaries AS

SELECT EmployeeID, Name, Salary

FROM Employees

WHERE Department = 'IT';
```

☑Update salary using the view:

sql

CopyEdit

```
UPDATE UpdateEmployeeSalaries

SET Salary = Salary + 5000

WHERE Name = 'Bob';
```

 The update **reflects in the underlying Employees table**.

 Note:

- A view can be updated only if it **does not contain** `JOIN`, `GROUP BY`, or aggregate functions.
- Indexed views can **improve performance** (see next section).

□ 6. Indexed View (Materialized View)

An **Indexed View** (Materialized View) **stores the results physically** for performance improvement.

Example: Create an Indexed View

sql

CopyEdit

```
CREATE VIEW IndexedAvgSalary

WITH SCHEMABINDING -- Prevents schema changes

AS

SELECT Department, COUNT(*) AS EmployeeCount, AVG(Salary) AS AvgSalary

FROM dbo.Employees

GROUP BY Department;
```

☑**Create a unique clustered index on the view:**

sql

CopyEdit

```
CREATE UNIQUE CLUSTERED INDEX IX_IndexedAvgSalary ON
IndexedAvgSalary(Department);
```

□ Indexed views **improve performance** but consume storage.

□ 7. Alter and Drop Views

☑**Modify a View (`ALTER VIEW`)**

sql

CopyEdit

```
ALTER VIEW EmployeeSalaries AS

SELECT EmployeeID, Name, Salary, Department

FROM Employees;
```

☑ Delete a View (`DROP VIEW`)

sql

CopyEdit

```
DROP VIEW EmployeeSalaries;
```

☑ Summary of Views

View Type	Example
Basic View	`CREATE VIEW EmployeeSalaries AS SELECT EmployeeID, Name, Salary FROM Employees;`
Filtered View	`CREATE VIEW IT_Employees AS SELECT * FROM Employees WHERE Department = 'IT';`
Join View	`CREATE VIEW EmployeeDetails AS SELECT e.Name, d.DepartmentName FROM Employees e JOIN Departments d ON e.DepartmentID = d.DepartmentID;`

Aggregation View	`CREATE VIEW AvgSalaryByDepartment AS SELECT Department, AVG(Salary) FROM Employees GROUP BY Department;`
Indexed View	`CREATE VIEW IndexedAvgSalary WITH SCHEMABINDING AS SELECT Department, COUNT(*) FROM Employees GROUP BY Department;`

—----------

Cursors in MSSQL with Examples

A **Cursor** in **Microsoft SQL Server (MSSQL)** is used to **fetch and process rows one by one** from a result set. Unlike `SELECT` queries that work with the entire result set at once, cursors allow **row-by-row** processing.

□ Why Use Cursors?

☑ **Process row-by-row** when set-based operations are not possible.
☑ **Perform complex row-wise calculations** that cannot be handled by `SELECT`.
☑ **Iterate over results** for procedural logic.

□ **Note:**
Cursors can be **slow and memory-intensive**, so use them **only when necessary**. Set-based operations (`JOINs`, `CTEs`, `WHILE`) are usually **faster and more efficient**.

□ Syntax of a Cursor

sql
CopyEdit

```
DECLARE CursorName CURSOR FOR
SELECT column1, column2 FROM TableName;
```

```
OPEN CursorName;
FETCH NEXT FROM CursorName INTO @Variable1, @Variable2;

WHILE @@FETCH_STATUS = 0
BEGIN
    -- Process the row
    FETCH NEXT FROM CursorName INTO @Variable1, @Variable2;
END;

CLOSE CursorName;
DEALLOCATE CursorName;
```

□ Example 1: Basic Cursor to Iterate Over Employees

Scenario:

You have an `Employees` table and want to print **each employee's name and salary**.

Step 1: Sample Employees Table

sql
CopyEdit

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(50),
    Salary INT
);

INSERT INTO Employees VALUES
(1, 'Alice', 60000),
(2, 'Bob', 50000),
(3, 'Charlie', 70000);
```

Step 2: Declare and Use a Cursor

sql
CopyEdit

```
DECLARE @EmployeeName VARCHAR(50), @Salary INT;

-- Declare the cursor
DECLARE EmployeeCursor CURSOR FOR
SELECT Name, Salary FROM Employees;

-- Open the cursor
OPEN EmployeeCursor;

-- Fetch the first row
FETCH NEXT FROM EmployeeCursor INTO @EmployeeName, @Salary;

-- Loop through all rows
WHILE @@FETCH_STATUS = 0
BEGIN
    PRINT 'Employee: ' + @EmployeeName + ' earns $' + CAST(@Salary AS
VARCHAR);

    -- Fetch the next row
    FETCH NEXT FROM EmployeeCursor INTO @EmployeeName, @Salary;
END;

-- Close and deallocate cursor
CLOSE EmployeeCursor;
DEALLOCATE EmployeeCursor;
```

☑Output in Messages Tab:

bash
CopyEdit

```
Employee: Alice earns $60000
Employee: Bob earns $50000
Employee: Charlie earns $70000
```

 The cursor **fetches each row, processes it, and moves to the next row**.

□ Example 2: Cursor with Update Operation

Scenario:

Increase the salary of employees earning less than **$60,000** by **$5,000**.

sql
CopyEdit

```
DECLARE @EmployeeID INT, @Salary INT;

-- Declare Cursor to Fetch Employees with Low Salary
DECLARE SalaryCursor CURSOR FOR
SELECT EmployeeID, Salary FROM Employees WHERE Salary < 60000;

-- Open the Cursor
OPEN SalaryCursor;

-- Fetch the first row
FETCH NEXT FROM SalaryCursor INTO @EmployeeID, @Salary;

-- Loop through all rows
WHILE @@FETCH_STATUS = 0
BEGIN
    -- Update the Salary
    UPDATE Employees SET Salary = Salary + 5000 WHERE EmployeeID =
@EmployeeID;

    PRINT 'Updated Salary for Employee ID: ' + CAST(@EmployeeID AS
VARCHAR);

    -- Fetch the next row
    FETCH NEXT FROM SalaryCursor INTO @EmployeeID, @Salary;
END;

-- Close and Deallocate Cursor
```

```
CLOSE SalaryCursor;
DEALLOCATE SalaryCursor;
```

☑**Result:**
The salaries of employees earning **less than $60,000** are increased by **$5,000**.

☑**Updated Data:**

pgsql
CopyEdit

```
EmployeeID | Name    | Salary
-----------|---------|-------
1          | Alice   | 60000
2     |Bob   |55000 ☑(Increased)
3          | Charlie | 70000
```

 The cursor **iterates through employees and updates salaries conditionally**.

Example 3: Cursor with Multiple Variables

Scenario:

Fetch **Employee Name, Salary, and Department**.

sql
CopyEdit

```
DECLARE @EmployeeName VARCHAR(50), @Salary INT, @Department
VARCHAR(50);

-- Declare Cursor
DECLARE EmployeeCursor CURSOR FOR
SELECT Name, Salary, Department FROM Employees;

-- Open Cursor
OPEN EmployeeCursor;
```

```
FETCH NEXT FROM EmployeeCursor INTO @EmployeeName, @Salary,
@Department;

WHILE @@FETCH_STATUS = 0
BEGIN
    PRINT @EmployeeName + ' works in ' + @Department + ' department
with salary $' + CAST(@Salary AS VARCHAR);

    -- Fetch next row
    FETCH NEXT FROM EmployeeCursor INTO @EmployeeName, @Salary,
@Department;
END;

-- Close and Deallocate Cursor
CLOSE EmployeeCursor;
DEALLOCATE EmployeeCursor;
```

☑**Output in Messages Tab:**

javascript
CopyEdit

```
Alice works in HR department with salary $60000
Bob works in IT department with salary $50000
Charlie works in Finance department with salary $70000
```

 The cursor **fetches and prints multiple column values per row**.

Cursor Best Practices

☑**Always close and deallocate cursors** (`CLOSE` and `DEALLOCATE`).
☑**Use `SET BASED` operations (`UPDATE`, `JOIN`) instead** where possible.
☑**Use cursors only when absolutely necessary**, as they **consume memory** and can **impact performance**.

□ Alternative: Using a WHILE Loop Instead of Cursors

Instead of a cursor, you can often use a `WHILE` loop, which is **faster and more efficient**.

sql
CopyEdit

```
DECLARE @MinID INT, @MaxID INT;

SELECT @MinID = MIN(EmployeeID), @MaxID = MAX(EmployeeID) FROM
Employees;

WHILE @MinID <= @MaxID
BEGIN
    PRINT 'Processing Employee ID: ' + CAST(@MinID AS VARCHAR);

    -- Move to next employee
    SET @MinID = @MinID + 1;
END;
```

☑**This approach is often better than using a cursor.**

☑Summary of Cursors

Cursor Type	Description
Forward-Only Cursor	Default cursor, moves **one row at a time** in forward direction only.
Static Cursor	Creates a **temporary snapshot** of data, does not reflect changes.
Dynamic Cursor	Reflects **real-time updates** in the data.
Keyset-Driven Cursor	Tracks changes to the key values but not other modifications.
